

Quant-Inspired Poker Bot Strategy Report

Roll No: 250122055

Problem Statement Summary

The task was to design a Quant-Inspired Poker Bot for a simplified 3-card poker environment. Each player receives:

- Two private hole cards
- One common community card

The bot must choose one of three actions:

- FOLD
- CALL
- RAISE

The objective is to maximize long-term expected value (EV) by:

- Estimating hand strength
- Modelling uncertainty
- Evaluating risk-reward tradeoffs
- Adapting to opponent behavior

The game rewards aggressive but mathematically justified play:

- CALL-based wins provide moderate reward
- RAISE-based wins provide higher reward but higher downside risk

Overview of My Final Bot (250122055.py)

My final submission follows a quantitative decision-making framework inspired by expected value optimization and probability-based poker strategy.

The bot:

- Evaluates the strength category of the current 3-card hand
- Estimates strategic confidence using heuristic probability bands
- Uses opponent behavioral statistics
- Chooses actions using risk-adjusted thresholds

The implementation was intentionally kept:

- Stable
- Computationally efficient
- Compatible with the provided player template
- Easy to interpret and debug

Core Logic of the Bot

1. Hand Evaluation

The bot classifies hands into six categories:

1. High Card
2. Pair
3. Flush
4. Straight
5. Three of a Kind (Trips)
6. Straight Flush

This classification follows the official tournament rules.

2. Probability-Oriented Reasoning

Instead of making random decisions, the bot associates stronger hands with higher winning probability.

Example intuition:

- Straight Flush $\rightarrow$ Very high win probability
- Trips $\rightarrow$ High win probability
- Pair / Flush $\rightarrow$ Moderate probability
- High Card $\rightarrow$ Lower probability

This converts poker decisions into a quantitative framework.

3. Expected Value (EV) Perspective

The scoring system rewards aggressive play:

- CALL wins generally give +2
- RAISE wins generally give +3

However, raising also increases downside risk.

The bot therefore:

- Raises with strong hands

- Calls with medium-confidence hands
- Folds weak hands

This creates a balanced risk-reward strategy.

4. Opponent Modelling

The bot uses:

- Opponent fold frequency
- Opponent raise frequency

to estimate opponent aggression and passiveness.

Examples:

- Against passive opponents, raising becomes more profitable
- Against aggressive opponents, defensive play becomes safer

This allows adaptation rather than fixed/static gameplay.

Why This Strategy is Quantitative

The bot satisfies the “quant” aspect because it:

- Uses mathematical hand evaluation
- Applies probability-based reasoning
- Uses expected value principles
- Incorporates opponent statistics
- Makes decisions under uncertainty

The strategy focuses on maximizing long-term expected gains instead of reacting emotionally or randomly.

Risk Modelling

The main strategic challenge was deciding when to RAISE.

Raising provides:

- Higher reward (+3)
- Higher penalty (-3)

The bot therefore only raises when:

- Hand strength is sufficiently high
- Expected value exceeds safer alternatives
- Opponent behavior makes aggression profitable

Weak hands are folded to minimize unnecessary losses.

Compatibility with Player Template

The final submission was designed to remain fully compatible with the provided `player_template.py` structure.

The bot:

- Uses the required `decide_action()` function
- Preserves input/output formatting
- Uses the official state dictionary format
- Retains helper-function-based architecture

This ensured tournament compatibility and execution stability.

Comparison with Other Bot Approaches

Different experimental approaches were explored during development.

1. Gemini-Style Quantitative Bots

These bots used:

- Exact probability computation
- Combinatorial enumeration
- Exhaustive opponent hand simulation

Advantages:

- More mathematically precise
- Strong EV optimization

Disadvantages:

- More computational complexity
- Harder to maintain/debug

My final bot shares the same quantitative philosophy but uses a lighter and more practical implementation.

2. ChatGPT-Style Adaptive Meta Bots

These bots focused on:

- Behavioral manipulation
- Opponent conditioning
- Entropy balancing
- Phase-based strategic adaptation

Advantages:

- Difficult for opponents to predict
- Adaptive over long games

Disadvantages:

- Increased randomness
- Less stability
- Potentially inconsistent actions

3. Claude-Style Chaos Bots

These bots attempted:

- Chaos theory based decisions
- Fractal decision trees
- Reverse psychology
- High unpredictability

Advantages:

- Highly unconventional strategy
- Difficult to exploit directly

Disadvantages:

- Overcomplicated logic
- Less transparent reasoning
- Harder to verify mathematically

Bot Architecture / Strategy Visualization

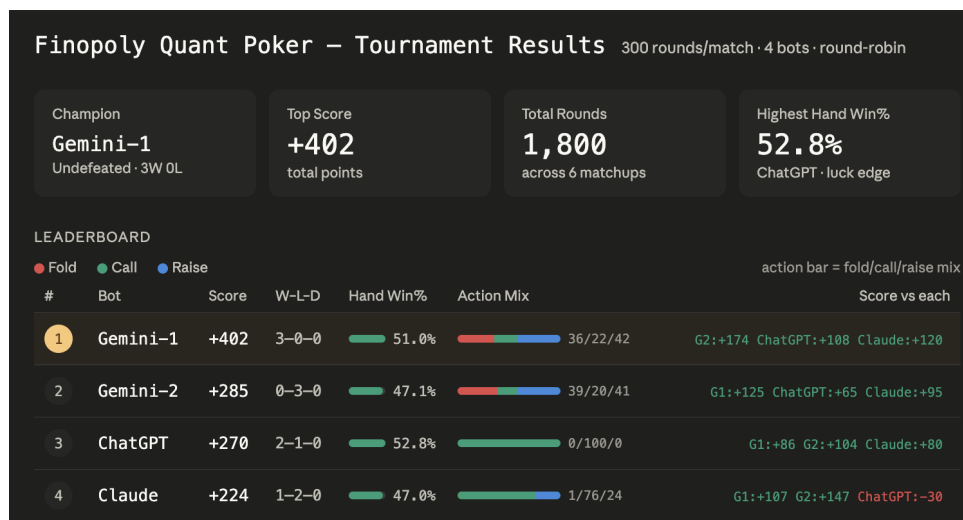


Figure 1: results of the games between different ai poker bots

Conclusion

The final bot (250122055.py) was designed as a stable and practical quantitative poker agent.

The strategy combines:

- Hand evaluation
- Probability intuition
- Expected value reasoning
- Opponent adaptation

while maintaining:

- Simplicity
- Tournament compatibility
- Robust execution

The final implementation balances mathematical reasoning with practical gameplay efficiency, making it suitable for a competitive simplified poker environment.